Manual del programador — sistemaproyecto

Versión: 1.0.0

Fecha: 15 de octubre de 2026

Autor: Malek Tomas Cheheid Tubello

Materia: Laboratorio de Programación (LPR) — EEST N.º 1 "Eduardo Ader", Vicente López

Actividad: 5 — Modelado de datos con estructuras (struct) y punteros

Repositorio: https://github.com/malekcheheid-art/sistemaproyecto

Licencia: MIT

1. Descripción general

Este manual describe el funcionamiento interno del programa `src/main.cpp` de la Actividad 5. El programa modela una entidad del Proyecto Integrador Anual —un sensor de temperatura del aula— mediante una estructura de datos (`struct`) en C++ y demuestra cómo se controla el ingreso y la modificación de sus miembros por referencia, a través de punteros y del operador flecha (`->`).

Destinatarios: estudiantes de 5.° año que cursen Laboratorio de Programación y docentes que evalúen la entrega.

2. Requisitos

- Compilador `g++` (MSYS2/MinGW) en Windows.
- Visual Studio Code con terminal PowerShell (opcional, recomendado).
- Como alternativa, si la PC de la escuela no permite compilar por permisos: compilador en línea OnlineGDB, generando las capturas desde ahí.

3. Estructura del repositorio

```
sistemaproyecto/
|-- .gitignore                    Reglas de exclusión (.exe, .vscode/)
|-- LICENSE                       Licencia MIT escolar
|-- README.md                     Portada con integrantes y descripción del modelo
|-- docs/
|   |-- InformeEEST1_LPR2026_ACT05_G03_v1.0.0.pdf   Informe APA v7
|   `-- manuales/
|       |-- manual_programador_v1.0.0.pdf           Este manual
|       `-- manual_programador_v1.0.0.md            Fuente de este manual
|-- src/
|   `-- main.cpp                  Programa principal modular
`-- capturas/
    `-- ejecucion_struct.png      Evidencia de ejecución
```

4. Modelo de datos: EntidadProyecto

La entidad modelada agrupa, bajo un mismo nombre lógico, los tres datos de un sensor del prototipo:

```
struct EntidadProyecto {
    int   id;          // Identificador numérico del componente
    char  nombre[50];  // Nombre o descripción del componente
    float metrica;     // Lecturas de sensores, consumo, avance, etc.
};
```

- `id`: entero que identifica al sensor dentro del sistema.
- `nombre[50]`: cadena de caracteres de longitud fija (máximo 49 útiles + `\0`).
- `metrica`: valor decimal, por ejemplo la temperatura en grados centígrados.

Los tres campos se almacenan en bloques contiguos de memoria RAM. Con esta definición, el tamaño total de la estructura es de 60 bytes (`sizeof(EntidadProyecto)` = 60, verificado al compilar), por lo que copiarla por valor en cada llamada a función resultaría innecesariamente costoso (ver apartado 7.2).

5. Funciones del programa

5.1 `main()`

Declara la variable `miEntidad` en la pila (*stack*) con valores de seguridad (`{0, "Vacio - Malek Cheheid", 0.0f}`), muestra el encabezado del programa y llama a las funciones pasando la **dirección de memoria** de la estructura con el operador `&miEntidad`.

5.2 `cargarDatos(EntidadProyecto* ptr)`

Recibe un puntero a la estructura y escribe los datos del usuario usando el operador flecha:

```
void cargarDatos(EntidadProyecto* ptr) {
    cout << "=> Ingrese el ID de la entidad (entero): ";
    cin >> ptr->id;

    // Limpieza del buffer obligatoria antes de cin.getline,
    // para eliminar el Enter remanente.
    cin.ignore();

    cout << "=> Ingrese el Nombre o Descripcion: ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";
    cin >> ptr->metrica;
}
```

Cada acceso con `ptr->campo` equivale a `(*ptr).campo`: primero se desreferencia el puntero y después se accede al miembro.

5.3 `mostrarDatos(const EntidadProyecto* ptr)`

Recibe el puntero como `const`, es decir, en modo solo lectura. Esto le avisa al compilador que la función no debe modificar la estructura, y genera un error si se intenta hacerlo. Imprime los tres miembros y, por último, la dirección de memoria real de la struct (`&(*ptr)`), lo que prueba que el puntero trabaja sobre la variable original.

6. Compilación y ejecución

Compilar desde la raíz del repositorio (PowerShell):

```
# 1. Compilar el programa generando el ejecutable en src/
g++ src/main.cpp -o src/sistemaproyecto.exe

# 2. Ejecutar la aplicación en Windows
.\src\sistemaproyecto.exe
```

> *Nota: el ejecutable* `.exe` *está excluido del control de versiones por el archivo* `.gitignore`*; solo se versiona el código fuente.*

7. Conceptos clave aplicados

7.1 Operador flecha (`->`)

Es la sintaxis exclusiva de C++ para acceder a los campos de una estructura a través de un puntero. `ptr->id` es exactamente lo mismo que `(*ptr).id`.

7.2 Pasaje por referencia

Enviar la dirección de memoria (`&`) de un objeto a una función permite modificar el original sin crear duplicados en la RAM. En un sistema de 64 bits solo se transfieren 8 bytes (la dirección) en lugar de copiar los ~58 bytes de la estructura completa en cada llamada.

7.3 Buffer de entrada y `cin.ignore()`

Cuando se lee con `cin >>`, la tecla Enter no se consume: el `\n` queda en el buffer. Si después se ejecuta `cin.getline`, esa función lee ese salto de línea como primer carácter y termina de inmediato, por lo que el texto parece omitirse. `cin.ignore()` descarta el carácter sobrante y corrige el problema.

8. Caso de prueba

Datos de entrada utilizados para la evidencia del repositorio:

Dato	Valor
ID de la entidad	7
Nombre o descripción	Sensor de temperatura - aula 3B
Métrica de operación	27.5

Salida esperada: el programa vuelve a mostrar los tres valores cargados y la dirección de memoria de la estructura. La ejecución se documenta en `capturas/ejecucion_struct.png`.

9. Cómo extender el modelo

Para modelar más sensores del proyecto se puede reutilizar la misma estructura agregando un campo por cada magnitud medida, por ejemplo:

```
struct EntidadProyecto {
    int    id;
    char   nombre[50];
    float  metrica;      // lectura principal
    char   unidad[10];   // p. ej. "Celsius"
    bool   activo;       // estado del sensor
};
```

Para manejar varios sensores a la vez se puede declarar un arreglo de estructuras (`EntidadProyecto sensores[10];`) y recorrerlo con un bucle, aplicando los mismos conceptos de pasaje por referencia.

10. Solución de problemas frecuentes

- **El programa se salta el ingreso del nombre:** falta la limpieza del buffer con `cin.ignore()` después de la lectura del ID.
- **Se muestran caracteres raros en la consola:** el código fuente está escrito sin acentos para evitar problemas de codificación en la terminal de Windows.
- **Error de compilación "was not declared in this scope":** la `struct` debe estar declarada antes de las funciones que la utilizan.

11. Control de versiones

El historial de desarrollo se mantiene con dos ramas: `development` para los avances y `main` para las versiones estables de entrega. Los commits son descriptivos, por ejemplo `creo el main.cpp` o `subo la captura`.

12. Licencia

Distribuido bajo licencia MIT. Consultar el archivo `LICENSE` en la raíz del repositorio.